From an Unused Laptop to a Home Server

Building a Self-Hosted Infrastructure Lab for Learning Deployment and DevOps

Technical Case Study • Current phase completed: October 2026

1. Executive Summary

Instead of leaving an old Lenovo ThinkPad T420 unused, I repurposed it as a personal home server. The goal was not to build a production system. It was to create a hands-on learning environment where I can understand what happens after code is written: operating-system preparation, networking, databases, message brokers, containers, service discovery, configuration, and troubleshooting.

The server now runs a Docker-based environment with PostgreSQL and Apache Kafka on a dedicated Docker network, and it can host and run Spring Boot applications. It is intentionally home-network only, and it is the foundation for the next phase: an automated CI/CD pipeline using GitHub Actions.

2. Why Repurpose an Old Laptop?

An unused laptop can become a useful infrastructure laboratory. A cloud VM is convenient, but a local server makes the underlying hosting process visible and safe to experiment with.

- Reuse existing hardware instead of leaving it idle, at zero additional cost.
- Learn Linux server administration on real hardware.
- Understand Docker networking and service discovery instead of treating containers as black boxes.
- Run databases and infrastructure locally without paying for cloud resources during experimentation.
- Break things, troubleshoot them, rebuild them, and learn from the process.
- Create a foundation for CI/CD, monitoring, personal cloud storage, and future projects.

3. Server Hardware and Operating Environment

Component	Configuration
Machine	Lenovo ThinkPad T420
CPU	Intel Core i5-2520M, 2 cores / 4 threads
Memory	8 GiB RAM
Storage	750 GB 7200 RPM SATA HDD
Operating System	Linux Mint 22.3 Zena (Ubuntu 24.04 base)
Kernel	7.0.0-38-generic
Primary network	Wi-Fi (Intel Centrino Advanced-N 6205)

The hardware is modest by modern standards, which is exactly what makes it valuable for learning resource-aware deployment. Lightweight services run comfortably at 8 GiB RAM, while heavier infrastructure such as Kafka has to be managed thoughtfully.

4. Current Architecture

The environment is organized around Docker, with each service isolated in its own container and connected through a dedicated bridge network:

- Linux Mint runs directly on the ThinkPad.
- Docker Engine and the Docker Compose plugin provide the container runtime.
- PostgreSQL 17 runs in a container with persistent storage.
- Apache Kafka 4.0.1 runs in a container as a single-node development broker (KRaft mode).
- Spring Boot applications run as containers on the same network and are exposed to devices on the home network.

```
HOME SERVER
Linux Mint -> Docker -> dedicated bridge network
  |-- Spring Boot application containers  (port 8080)
  |-- PostgreSQL 17                       (port 5432)
  `-- Apache Kafka 4.0.1                  (port 9092)
```

5. Server Preparation

The machine was updated and prepared as a development server. Java 21 and Maven were installed so applications can be built and diagnosed directly on the host when needed.

- OpenJDK 21 with JAVA_HOME configured.
- Apache Maven 3.9 for building Java projects.
- Docker Engine and the Docker Compose plugin for running services.

A Docker permission issue was also encountered and resolved by adding the Linux user to the docker group and refreshing the shell session. This was an important learning point: Docker CLI access depends on permission to communicate with the Docker daemon socket, so a working installation does not automatically mean every user session can use it.

6. Core Services

PostgreSQL 17. Deployed as a Docker container with persistent storage kept outside the container lifecycle. Because the database is separate from the application containers, rebuilding or replacing an application does not require rebuilding the database or losing its data.

Apache Kafka 4.0.1. Deployed in KRaft mode as a single-node development broker and attached to the same Docker network, so applications can resolve the broker by its container name. Kafka reached the RUNNING state and is the most resource-sensitive service on this hardware.

7. Docker Networking: The Key Concept

One of the most useful lessons was the difference between localhost on the host machine and localhost inside a container.

When an application was first moved into a container, its configuration still pointed to localhost for PostgreSQL and Kafka. Inside a container, localhost refers to that container itself, not to the other services, so the connections failed. The fix was to use Docker service names instead:

```
PostgreSQL ->  jdbc:postgresql://postgres:5432/<database>
Kafka      ->  kafka:9092
```

Containers on the same Docker network can reach each other by container or service DNS names. This is a fundamental containerization concept and explains how multi-service systems communicate.

8. Configuration and Secrets

Deployment-specific values such as database credentials and token signing secrets are supplied through environment variables rather than hard-coded in source configuration. This keeps secrets out of the repository and allows the same image to run in different environments. Secrets should never be committed to GitHub; the CI/CD phase will use GitHub Actions secrets for the same reason.

9. Problems Encountered and What They Taught Me

Problem	Lesson
Docker permission denied on the daemon socket	Linux permissions and daemon access are part of real server administration.
Application could not connect to the database from a container	localhost is scoped to the current container; use the Docker network and service names.
Confusing secondary framework errors (e.g., Hibernate dialect)	Later errors often originate from an earlier infrastructure failure. Find the root cause first.
Configuration fixed in source, but the container still used old settings	Changing source code does not change an already-built artifact. Rebuild the image and recreate the container.
Limited RAM, older CPU and HDD	Self-hosting requires knowing which services are heavy and avoiding unnecessary always-on infrastructure.

10. What This Project Demonstrates

- Linux server administration
- Docker and Docker Compose
- Container networking and service discovery
- PostgreSQL deployment and persistent storage

- Kafka infrastructure on constrained hardware
- Environment-based configuration and secret handling
- Troubleshooting across OS, container, network and application layers
- Practical understanding of the hosting lifecycle

The most important outcome is the hands-on understanding of the path from source code to a running service: build, package, create a container image, configure dependencies, connect services through a network, expose the application, and diagnose failures when the environment does not match the configuration.

11. Next Phase: Automated CI/CD with GitHub Actions

The next phase removes the manual deployment steps and turns the laptop into a small self-hosted deployment platform. The planned workflow is:

1. Push code to GitHub.
2. Trigger a GitHub Actions workflow.
3. Build and test the application.
4. Build an updated Docker image.
5. Connect securely to the home server (credentials stored as GitHub Actions secrets).
6. Pull the new image and recreate only the application container.
7. Keep PostgreSQL and Kafka running independently, so data and messaging are unaffected by deployments.

This provides practical experience with CI/CD concepts before moving to larger cloud infrastructure.

12. Roadmap: Future Improvements

Improvement	Purpose
GitHub Actions automated deployment	Fully automated path from commit to running service.
Health checks and restart policies	Let services recover automatically from failures.
Reverse proxy	Host multiple applications behind one entry point.
Centralized logging and monitoring	Visibility into resource usage and service health.
Backup strategy	Protect database and application data.
Personal cloud storage	Self-hosted file storage as another service to operate.
More Spring Boot projects	Run several applications side by side on the same server.
Structured Docker Compose environments	Reproducible, version-controlled service definitions.
SSD upgrade and additional RAM	Improve I/O and give Kafka and other services more headroom.

13. LinkedIn Showcase Summary

Repurposed an old Lenovo ThinkPad T420 into a home server to learn real-world application hosting and deployment. Built a Docker-based environment with PostgreSQL and Apache Kafka on a dedicated Docker network, running on Linux Mint. Along the way I worked through Linux permissions, container networking, environment-based secrets and deployment troubleshooting. The next phase is automated CI/CD with GitHub Actions, followed by monitoring, a reverse proxy and backups. The project shows that even unused hardware can become a valuable infrastructure and DevOps learning platform.

14. Closing Note

This project started with an old laptop that was no longer being used. Instead of treating the hardware as obsolete, I turned it into a practical server and learning environment. The value is in the engineering understanding gained from building, breaking and troubleshooting the infrastructure end to end.